

INFORME TECNICO DE ARQUITECTURA

Fitness Microservices System

Este documento presenta una guía completa del sistema de microservicios para la plataforma de Fitness. Explica la arquitectura, los mecanismos internos implementados, como levantar el entorno y como realizar las pruebas de funcionamiento de extremo a extremo.

1. Arquitectura y Componentes del Sistema

El sistema esta diseñado bajo una arquitectura de microservicios descentralizada apoyada en el ecosistema de Spring Cloud y asegurada con OAuth2 (Keycloak).

Diagrama de Arquitectura de Microservicios

(Ver diagrama de flujo visual interactivo en la version Markdown: C:\Users\mauri\geminil\antigravity\brain\92ca0d4d-2fec-42e4-a52a-382ae7025c50\project_report.md)

Puertos y Modulos del Proyecto

Modulo	Puerto	Descripcion	Base de Datos / Almacenamiento
`eureka-server`	`8761`	Servidor de Descubrimiento de Netflix Eureka.	En memoria
`config-server`	`8888`	Servidor de Configuracion Centralizada (Perfil nativo leyendo archivos)	Sistema de archivos
`api-gateway`	`8080`	Puerta de entrada unificada. Enruta y valida los tokens de seguridad.	En memoria
`usuario-service`	`8081`	Gestiona usuarios y membresias.	MySQL (`usuario_db`)
`rutina-service`	`8082`	Gestiona rutinas de entrenamiento y ejercicios.	MySQL (`rutina_db`)
`progreso-service`	`8083`	Registra el historico de peso y avance fisico.	MySQL (`progreso_db`)
`keycloak`	`8180`	Servidor de identidad y acceso OAuth2.	H2 integrada
`mysql`	`3306`	Servidor de bases de datos relacionales.	Persistencia en Docker volume

2. Como Funciona el Sistema por Detras

A. Configuracion Centralizada (Spring Cloud Config)

Cada microservicio arranca con un archivo de propiedades minimalista que solo le indica al framework donde esta el servidor de configuracion:

```
spring.config.import=optional:configserver:http://config-server:8888
```

Al iniciar, el microservicio consulta al config-server, el cual lee las propiedades especificas (como contraseñas, URLs de bases de datos y seguridad) desde la carpeta centralizada config-repo.

B. Descubrimiento de Servicios (Netflix Eureka)

Cada servicio de negocio se registra automaticamente en Eureka usando su nombre (spring.application.name).

Cuando un servicio como rutina-service necesita llamar a usuario-service, no usa direcciones IP ni puertos hardcoded; en su lugar, utiliza el nombre logico:

```
@FeignClient(name = "usuario-service")
```

Eureka se encarga de resolver la direccion IP y el puerto de forma dinamica, permitiendo balancear la carga de forma automatica (lb://usuario-service).

C. Seguridad Descentralizada (Keycloak OAuth2)

El sistema utiliza un flujo de Resource Server:

- El cliente pide un token de acceso a Keycloak enviando usuario y contraseña.
- Cada vez que hace una peticion a traves del API Gateway, adjunta el token JWT en la cabecera Authorization: Bearer <TOKEN>.
- El API Gateway y los microservicios validan de forma autonoma la firma digital y fecha de expiracion del token consultando la clave publica de Keycloak (jwk-set-uri), protegiendo todos los endpoints de negocio sin afectar el rendimiento.

D. Interceptor Feign para Propagacion de Tokens

Cuando rutina-service o progreso-service necesitan consultar a usuario-service a traves de Feign, deben estar autorizados.

Para resolver esto de forma limpia, se configuro un FeignClientInterceptor en ambos servicios:

```
@Component
public class FeignClientInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        ServletRequestAttributes attributes = (ServletRequestAttributes) RequestContextHolder.getRequestAttributes();
        if (attributes != null) {
            String authHeader = attributes.getRequest().getHeader("Authorization");
            if (authHeader != null && authHeader.startsWith("Bearer ")) {
                template.header("Authorization", authHeader);
            }
        }
    }
}
```

Este interceptor extrae automaticamente el token JWT del usuario que inicio la peticion y lo inyecta en la cabecera de la llamada interna Feign, permitiendo que la validacion inter-servicio sea exitosa.

3. Guia de Ejecucion del Proyecto

Requisitos Previos

- Docker Desktop instalado y en funcionamiento.
- Maven y JDK 17 o superior instalados (si deseas compilar localmente).

Instrucciones de Arranque en Docker

- Abre tu terminal de Windows (PowerShell o CMD).
- Navega a la carpeta del proyecto:


```
powershell
cd "C:\Users\mauri\Downloads\fitness-microservices\fitness-microservices"
```
- Compila los empaquetados jar de los microservicios:


```
powershell
mvn clean package -DskipTests
```

- Levanta el entorno multi-contenedor:
powershell
docker-compose up --build -d

Enlaces de Verificacion y Diagnostico

Una vez que el comando haya terminado, puedes abrir el navegador y visitar:

- Eureka Dashboard: <http://localhost:8761> (Verifica que aparezcan los servicios registrados).
- Consola de Keycloak: <http://localhost:8180> (Administrador: admin / admin).
- Configuraciones de Usuario: <http://localhost:8888/usuario-service/default>.

4. Guia Completa de Pruebas de APIs

Paso 1: Autenticarse en Keycloak para obtener el Token

Antes de llamar a cualquier API protegida, debes obtener el token de acceso. En Postman, crea la siguiente peticion:

- Metodo: POST
- URL: <http://localhost:8180/realms/fitness-realm/protocol/openid-connect/token>
- Headers: Content-Type: application/x-www-form-urlencoded
- Body (x-www-form-urlencoded):
 - * client_id : fitness-client
 - * client_secret : fitness-client-secret
 - * grant_type : password
 - * username : admin
 - * password : admin123
- *Copia todo el valor del campo access_token de la respuesta JSON.*

Paso 2: Crear un Usuario (Servicio de Usuarios)

Enviamos los datos del usuario al servicio a traves de la ruta del API Gateway.

- Metodo: POST
- URL: <http://localhost:8080/api/usuarios>
- Headers:
 - * Authorization: Bearer <PEGA_EL_TOKEN_AQUI>
 - * Content-Type: application/json
- Body (JSON):

```
json
{
  "nombre": "Carlos Mendoza",
  "email": "carlos.m@mail.com",
  "telefono": "987654321",
  "fechaRegistro": "2026-07-27"
}
```

- *Guarda el campo idUsuario devuelto en la respuesta (por ejemplo: 1).*

Paso 3: Asignar una Membresia (Servicio de Usuarios)

- Metodo: POST
- URL: http://localhost:8080/api/membresias
- Headers:
 - * Authorization: Bearer <PEGA_EL_TOKEN_AQUI>
 - * Content-Type: application/json
- Body (JSON):

```
json
{
  "idUsuario": 1,
  "tipoMembresia": "Premium Anual",
  "costo": 350.00,
  "fechaInicio": "2026-07-27",
  "fechaFin": "2027-07-27",
  "estado": "Activo"
}
```

Paso 4: Crear una Rutina de Ejercicios (Servicio de Rutinas)

Este paso desencadena la comunicacion Feign. rutina-service llamara por detras a usuario-service para verificar que el usuario con idUsuario = 1 sea real.

- Metodo: POST
- URL: http://localhost:8080/api/rutinas
- Headers:
 - * Authorization: Bearer <PEGA_EL_TOKEN_AQUI>
 - * Content-Type: application/json
- Body (JSON):

```
json
{
  "nombreRutina": "Rutina Hipertrofia Pecho/Triceps",
  "idUsuario": 1
}
```

- Prueba de error (Validacion): Si intentas enviar la peticion con un "idUsuario": 999 (ID de usuario inexistente), el microservicio respondera automaticamente con un error 404 Not Found y el mensaje "No existe el usuario con id: 999", confirmando que el flujo de verificacion y manejo de excepciones inter-servicio funciona a la perfeccion.

Paso 5: Registrar el Progreso Fisico (Servicio de Progresos)

- Metodo: POST

- URL: `http://localhost:8080/api/progresos`
- Headers:
 - * Authorization: Bearer <PEGA_EL_TOKEN_AQUI>
 - * Content-Type: `application/json`
- Body (JSON):

```
json
{
  "idUsuario": 1,
  "fechaRegistro": "2026-07-27",
  "peso": 78.50,
  "porcentajeGrasa": 15.4,
  "porcentajeMusculo": 42.1
}
```

Paso 6: Consultar el Historial de Progreso

- Metodo: GET
- URL: `http://localhost:8080/api/progresos/usuario/1`
- Headers:
 - * Authorization: Bearer <PEGA_EL_TOKEN_AQUI>
- Resultado: Devolvera el listado del historial de avance fisico de Carlos Mendoza.

5. Mantenimiento y Apagado Limpio

Para apagar la infraestructura cuando hayas terminado las pruebas, ejecuta:

```
docker-compose down
```

Si deseas eliminar tambien los datos persistidos en la base de datos MySQL para iniciar con tablas vacias en la siguiente ejecucion, anade la bandera `-v`:

```
docker-compose down -v
```